



CAHIER DES CHARGES

Projet Final du 2^{ème} quadrimestre

Système de contrôle automatique d'un ascenseur

Unités d'enseignement :

A208 — Acquisition et visualisation de données

A211 — Systèmes embarqués I

Classe : 2AU

Soumis le 30/05/2026

EL YAZAMI Yassine

TABICH Mohamed

2025-2026

Table des matières

1	Introduction	3
2	Carte mentale (Mind mapping)	3
3	Schéma blocs (Hardware).....	4
4	Croquis de la face avant (LabVIEW).....	4
5	Description du fonctionnement.....	5
5.1	Scénario général.....	5
5.2	Partie Systèmes embarqués (A211).....	6
5.3	Partie Acquisition & Visualisation (A208).....	7
5.4	Protocole de communication	8
5.5	Gestion des interruptions.....	9
5.6	Mémoire EEPROM I ² C.....	10
6	Historique des Révisions.....	11
7	Description du Fonctionnement – Version 2.0.....	12
7.1	Entrées Digitales.....	12
7.2	Capteurs	13
7.3	Sorties – LEDs d'état	13
7.4	Moteur – Driver L293D (remplace IRF520)	14
7.5	EEPROM I2C – NM24C08 (ajout V2.0).....	14
7.6	Complément EEPROM I ² C – Version 2.0.....	15
7.7	Communication UART – PIC ↔ LabVIEW (confirmation V2.0).....	15
7.7.1	Trames PIC → LabVIEW	15
7.7.2	Champs de la trame DATA	16
7.7.3	Trames LabVIEW → PIC	16
7.7.4	Justification des ajouts par rapport à la V1.0	17
7.8	Tableau Récapitulatif Complet du Brochage – Version 2.0.....	19
8	Historique des Révisions.....	21
9	Description du Fonctionnement – Version 3.0.....	22
9.1	Trame DATA – Format mis à jour	22
9.2	Trames PIC → LabVIEW	23
9.3	Trames LabVIEW → PIC – Tableau complet Version 3.0.....	24
9.4	Justification des ajouts et suppressions par rapport à la Version 2.0	25

1 Introduction

Dans le cadre du projet final du 2ème quadrimestre, qui regroupe les unités d'enseignement A208 (Acquisition et visualisation de données) et A211 (Systèmes embarqués I), nous avons choisi de réaliser un système de contrôle automatique d'ascenseur.

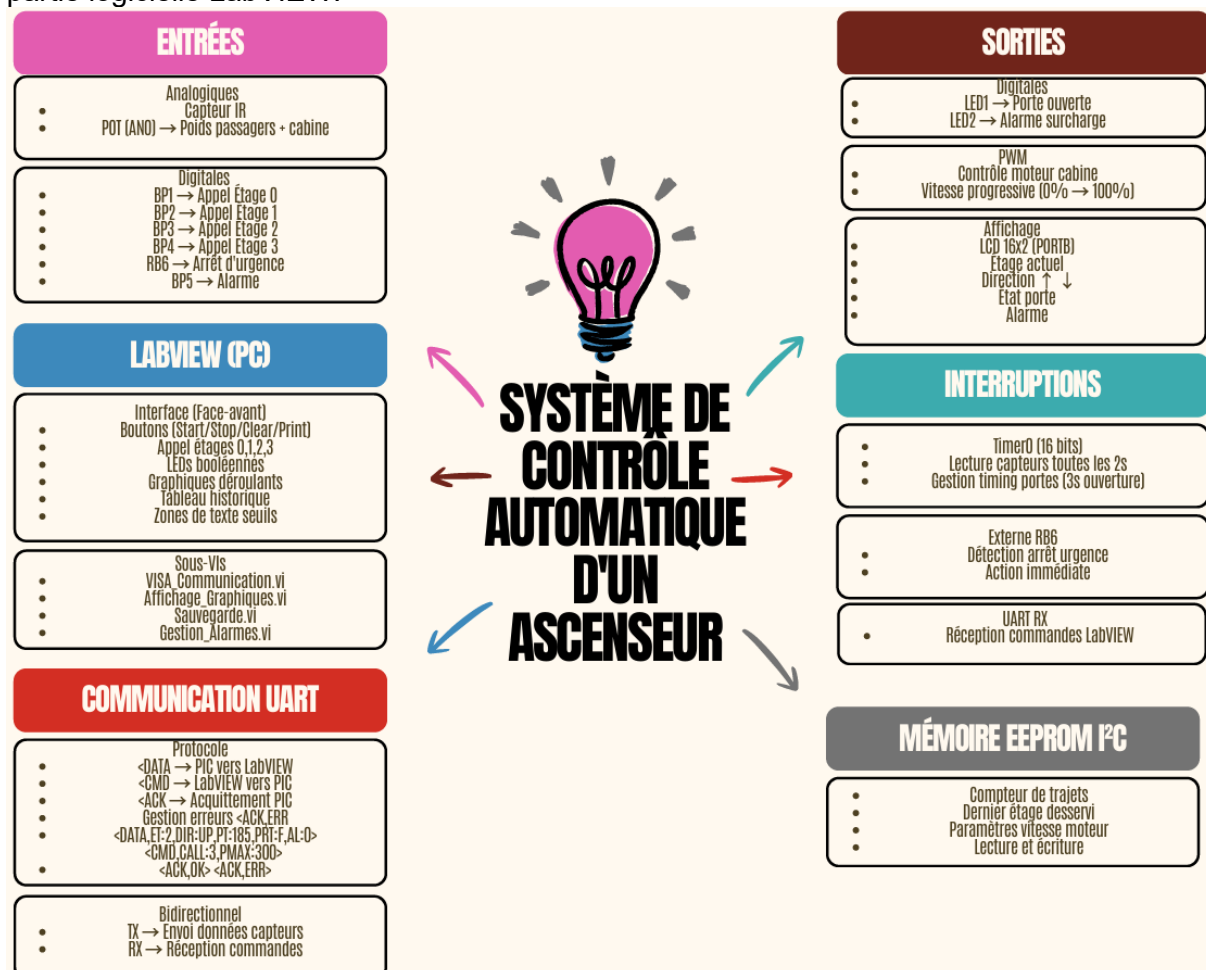
Ce projet s'inscrit dans le domaine de l'automatisation. L'ascenseur est un exemple concret du quotidien qui fait intervenir plusieurs fonctions automatisées : détection de présence, gestion des appels par étage, contrôle de la vitesse du moteur, surveillance du poids, gestion des alarmes et communication avec un opérateur via une interface informatique.

L'objectif principal est de simuler le fonctionnement d'un ascenseur avec le microcontrôleur PIC18F45K22, placé sur la carte EasyPIC V7 et LabVIEW sur PC. Les deux parties communiquent entre elles par une liaison série UART dans les deux sens, selon un protocole de communication (Visa).

Ce cahier des charges rassemble les choix techniques et fonctionnels retenus pour ce projet, ainsi que les différents éléments graphiques comme la carte mentale, le schéma blocs et le croquis de la face avant, afin de présenter la structure du système.

2 Carte mentale (Mind mapping)

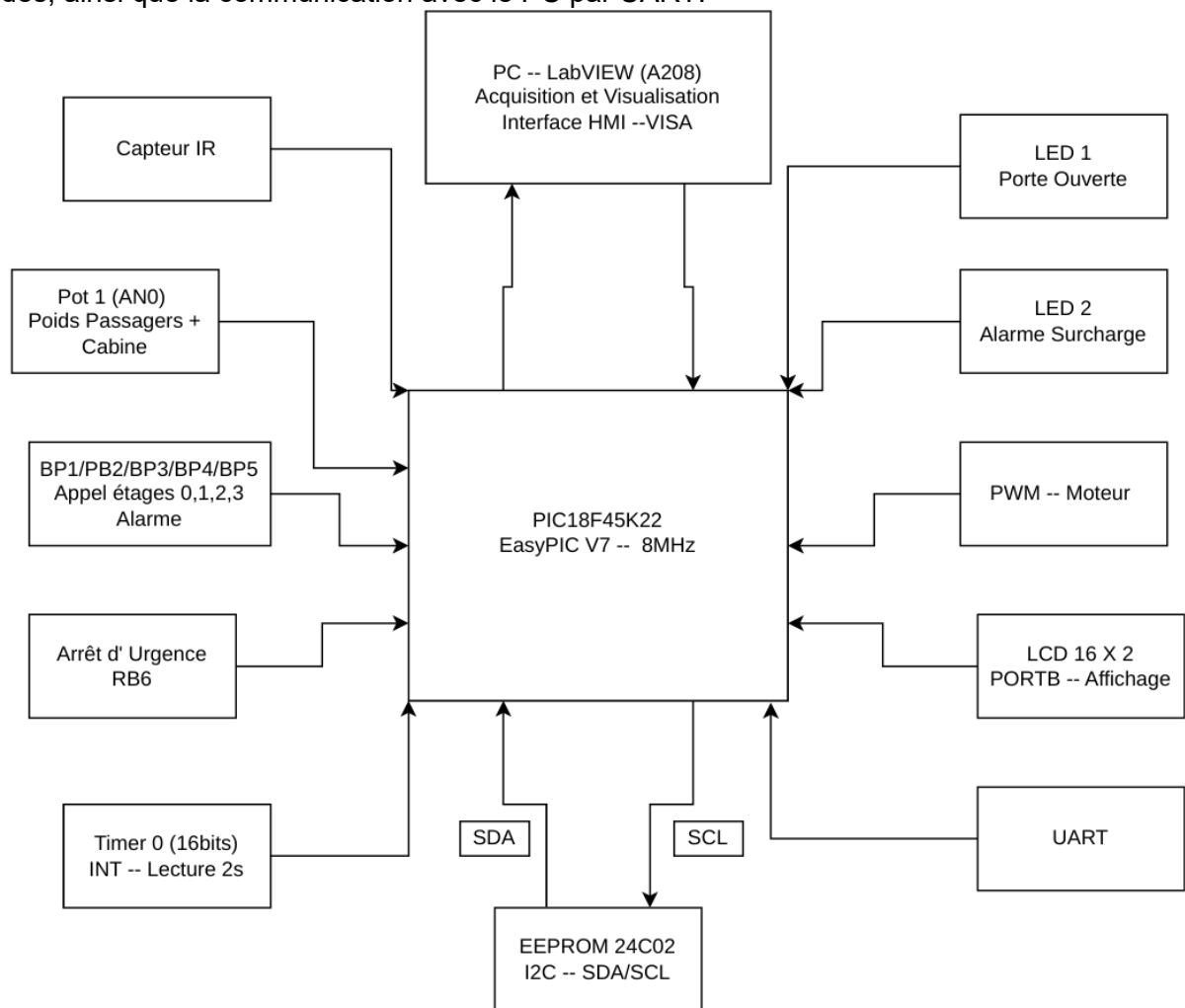
La carte mentale ci-dessous présente les différentes parties du projet ainsi que les liens entre elles. Elle s'organise autour du système complet, avec six branches principales : les entrées, les sorties, les interruptions, la mémoire EEPROM, la communication UART et la partie logicielle LabVIEW.



Cette carte mentale permet de voir rapidement l'ensemble des fonctions du système et la manière dont elles s'articulent. Chaque branche correspond à un point technique précis, ce qui aide à mieux comprendre le projet et à organiser le travail.

3 Schéma blocs (Hardware)

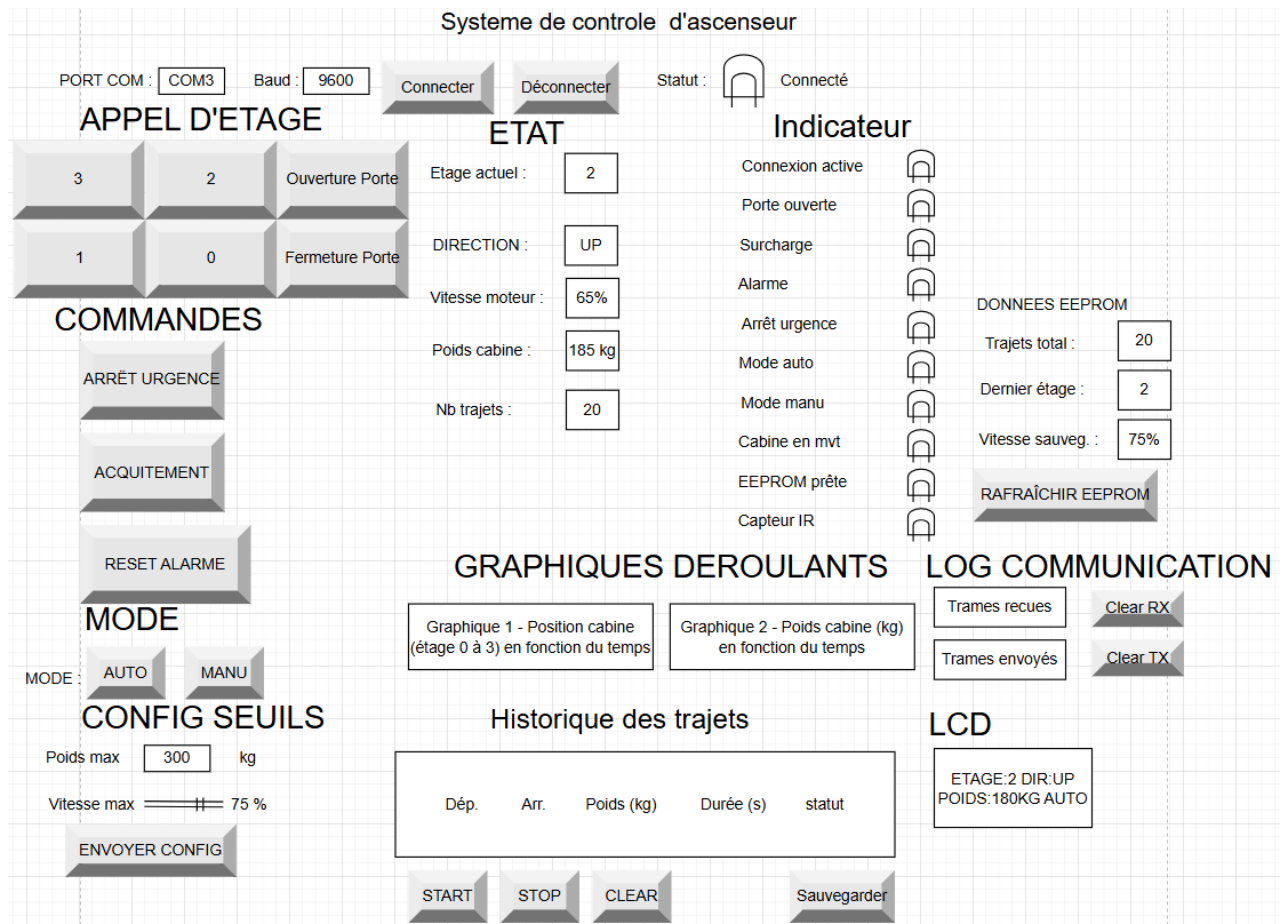
Le schéma fonctionnel ci-dessous présente l'organisation matérielle du système. Il montre les liens entre le microcontrôleur PIC18F45K22 et tous les périphériques qui lui sont raccordés, ainsi que la communication avec le PC par UART.



Le PIC18F45K22, placé sur la carte EasyPIC V7, est au centre du système. Il récupère les informations des capteurs et des boutons, traite ces données, pilote les actionneurs comme les LEDs, le moteur via PWM et le LCD, puis échange en continu avec l'application LabVIEW sur le PC.

4 Croquis de la face avant (LabVIEW)

Le croquis ci-dessous montre l'interface LabVIEW utilisée pour piloter le système et suivre son fonctionnement.



La face avant est organisée en plusieurs zones. En haut se trouve la connexion, avec le choix du port COM et l'état de la liaison. À droite, on regroupe les commandes de contrôle, comme les appels par étage, l'arrêt d'urgence, le reset alarme et les réglages du mode de fonctionnement. Au centre, les informations principales du système sont affichées, comme l'étage actuel, la direction, la vitesse, le poids et le nombre de trajets.

En bas, des graphiques permettent de suivre l'évolution de la cabine et du poids, tandis qu'un tableau garde l'historique des trajets. Sur la droite, une zone affiche les trames échangées entre le PIC et LabVIEW, ainsi que les données liées à la mémoire EEPROM.

5 Description du fonctionnement

5.1 Scénario général

Le projet simule un ascenseur à 4 étages, de l'étage 0 à l'étage 3. La cabine se déplace entre les étages en fonction des appels faits soit depuis les boutons physiques sur la carte, soit depuis l'interface LabVIEW sur le PC.

Le système fonctionne avec deux modes. En mode automatique, le PIC gère tout seul les déplacements, l'ouverture et la fermeture des portes, la surveillance du poids et les alarmes, sans intervention de l'opérateur. En mode manuel, l'opérateur prend la main depuis LabVIEW. Il peut ouvrir ou fermer la porte, démarrer ou arrêter le moteur, régler la vitesse avec le slider PWM, monter ou descendre d'un étage avec les boutons directionnels, tester les LEDs et remettre le compteur EEPROM à zéro.

En continu, le PIC envoie l'état du système vers LabVIEW toutes les 2 secondes grâce au Timer0. De son côté, LabVIEW peut envoyer des commandes au PIC à tout moment, et le PIC renvoie toujours un accusé de réception.

5.2 Partie Systèmes embarqués (A211)

La partie embarquée est développée en langage C à l'aide de l'IDE MikroC, sur le microcontrôleur PIC18F45K22 fonctionnant à 8 MHz, monté sur la carte EasyPIC V7.

- **Entrées analogiques**

Broche	Capteur	Rôle
AN0	Capteur infrarouge (IR)	Détecte la présence d'une personne devant la porte. Si une présence est détectée, la porte reste ouverte ou s'ouvre automatiquement.
AN1	Potentiomètre	Simule le poids total dans la cabine (passagers + bagages). Déclenche une alarme de surcharge si le seuil configuré est dépassé.

- **Entrées digitales**

Broche	Élément	Rôle
RB0	BP1	Appel de la cabine à l'étage 0.
RB1	BP2	Appel de la cabine à l'étage 1.
RB2	BP3	Appel de la cabine à l'étage 2.
RB3	BP4	Appel de la cabine à l'étage 3.
RB6	Bouton urgence	Déclenche un arrêt immédiat de la cabine via une interruption externe. LED2 s'allume et une alarme est envoyée à LabVIEW.
RB7	BP5	Alarme

- **Sorties digitales et affichage**

Élément	Connexion	Rôle
LED 1	RA0	Indique que la porte est ouverte.
LED 2	RA1	Indique une alarme active (surcharge ou arrêt d'urgence).
Sortie PWM	CCP1	Contrôle la vitesse du moteur de la cabine de 0% à 100%. La vitesse augmente progressivement au départ et diminue à l'approche de l'étage cible.
LCD 16x2	PORTB	Affiche : étage actuel, direction (UP/DOWN/STOP), poids cabine et mode en cours (AUTO/MANU).

5.3 Partie Acquisition & Visualisation (A208)

L'application PC est développée sous LabVIEW. Elle communique avec la carte EasyPIC V7 par le port série, avec la bibliothèque VISA. L'ensemble est organisé de façon modulaire, avec des sous-VIs pour les différentes fonctions du projet.

Sous-VI	Rôle
VISA_Communication.vi	Gère l'ouverture, la fermeture du port série et l'envoi/réception des trames UART. Vérifie les acquittements reçus.
Affichage_Graphiques.vi	Met à jour les deux graphiques déroulants (position cabine et poids). Gère l'échelle et l'historique affiché.
Sauvegardevi	Enregistre automatiquement dans un fichier texte chaque trajet effectué (départ, arrivée, poids, durée, statut).
Gestion_Alarmes.vi	Surveille les valeurs reçues et allume les indicateurs booléens correspondants (surcharge, urgence, capteur IR, mode...).

En mode manuel, les contrôles supplémentaires disponibles depuis la face avant sont :

- Ouvrir / Fermer la porte manuellement.
- Démarrer ou arrêter le moteur.
- Régler la vitesse du moteur via un slider (0% à 100%) envoyé par une trame <CMD>.

- Monter ou descendre d'un étage via les boutons directionnels.
- Tester les LEDs LED1 et LED2 individuellement.
- Remettre le compteur de trajets EEPROM à zéro.

En mode automatique, tous ces contrôles sont désactivés et c'est le PIC qui prend les décisions de manière autonome.

5.4 Protocole de communication

La communication entre le PIC et LabVIEW est bidirectionnelle. Nous avons défini un protocole simple basé sur des trames encadrées par les caractères < et >, ce qui permet de détecter facilement le début et la fin de chaque message, des deux côtés.

• Trames envoyées par le PIC vers LabVIEW

Trame	Description	Exemple
<DATA,...>	Envoyée toutes les 2 secondes (Timer0). Contient l'état complet du système.	<DATA,ET:2,DIR:UP,PT:185,PRT:F,AL:0>
<ACK,OK>	Acquittement positif : commande reçue et exécutée correctement.	<ACK,OK>
<ACK,ERR>	Acquittement négatif : trame reçue invalide ou corrompue.	<ACK,ERR>

• Champs de la trame DATA

Champ	Signification	Valeurs possibles
ET	Étage actuel de la cabine	0, 1, 2, 3
DIR	Direction du déplacement	UP, DOWN, STP (arrêté)
PT	Poids total dans la cabine (kg)	Valeur numérique (ex : 185)
PRT	État de la porte	O = ouverte, F = fermée

Champ	Signification	Valeurs possibles
AL	Alarme active	0 = aucune, 1 = active

- **Trames envoyées par LabVIEW vers le PIC**

Trame	Description	Exemple
<CMD,CALL:X>	Demande d'appel à un étage précis (X = 0 à 3).	<CMD,CALL:3>
<CMD,PMAX:X>	Envoi du poids maximum configuré (X en kg).	<CMD,PMAX:300>
<CMD,SPD:X>	Envoi de la vitesse moteur en mode manuel (X en %).	<CMD,SPD:75>
<CMD,DOOR:O>	Commande d'ouverture manuelle de la porte.	<CMD,DOOR:O>
<CMD,DOOR:F>	Commande de fermeture manuelle de la porte.	<CMD,DOOR:F>
<CMD,MOT:ON>	Démarrage du moteur en mode manuel.	<CMD,MOT:ON>
<CMD,MOT:OFF>	Arrêt du moteur en mode manuel.	<CMD,MOT:OFF>
<CMD,RST:EEP>	Remise à zéro du compteur de trajets en EEPROM.	<CMD,RST:EEP>
<CMD,GET:EEP>	Demande de lecture des données EEPROM.	<CMD,GET:EEP>

Après chaque commande reçue, le PIC renvoie toujours un accusé de réception sous la forme <ACK,OK> ou <ACK,ERR>. Si aucun retour n'arrive dans le délai prévu, LabVIEW considère que la communication est coupée et affiche une erreur dans le log.

5.5 Gestion des interruptions

Le système gère simultanément trois types d'interruptions, conformément aux exigences du projet :

Type	Source	Rôle dans le projet
Timer 0 (16 bits)	Interruption interne	Déclenché toutes les 2 secondes. À chaque déclenchement, le PIC lit les valeurs des capteurs (AN0 et AN1), met à jour le LCD et envoie une trame <DATA> vers LabVIEW. Il gère également le timing d'ouverture des portes (3 secondes).
Interruption externe	Bouton urgence RB6	Déclenché immédiatement lors d'un appui sur le bouton d'arrêt d'urgence. Le moteur s'arrête instantanément, LED2 s'allume et une trame <DATA> avec AL:1 est envoyée à LabVIEW sans attendre le prochain cycle du Timer0.
Interruption UART RX	Réception série RC7	Déclenché à chaque réception d'un caractère depuis LabVIEW. Le PIC assemble les caractères jusqu'à recevoir le délimiteur de fin >, analyse la trame <CMD> et exécute l'action correspondante. Renvoie ensuite un acquittement <ACK,OK> ou <ACK,ERR>.

5.6 Mémoire EEPROM I²C

La mémoire EEPROM externe est reliée au PIC par le bus I²C. Elle sert à conserver de façon permanente les données suivantes :

Adresse EEPROM	Donnée stockée	Mise à jour
0x00	Compteur total de trajets (2 octets)	Incrémenté à chaque trajet terminé.
0x02	Dernier étage desservi (1 octet)	Mis à jour à chaque arrêt à un étage.
0x03	Vitesse maximale configurée (1 octet, en %)	Mis à jour lors de la réception d'une commande <CMD,SPD:X>.
0x04	Poids maximum configuré (2 octets, en kg)	Mis à jour lors de la réception d'une commande <CMD,PMAX:X>.

Ces données peuvent être consultées depuis LabVIEW avec la commande <CMD,GET:EEP>. Le PIC lit alors les valeurs enregistrées dans l'EEPROM et les renvoie immédiatement dans une trame. Le compteur peut aussi être remis à zéro depuis LabVIEW avec la commande <CMD,RST:EEP>.

CDC Version 2.0

6 Historique des Révisions

Ce tableau liste tous les changements entre la Version 1.0 et la Version 2.0, avec la justification de chaque correction.

Version	Section modifiée	Changement apporté	Justification
V1.0	–	Version initiale	Rédaction initiale du cahier des charges
V2.0	Entrées – Boutons étages	BP1-BP4 → RD0, RD1, RD2, RD3	Correction brochage : les boutons sont sur PORTD, pas PORTA. Confirmé sur schéma Proteus.
V2.0	Entrées – Bouton ACQ	ACQ → RD4	Brochage incorrect en V1.0. Confirmé sur schéma Proteus.
V2.0	Capteur IR – Porte	IR sur RA0 en entrée digitale. Bouton poussoir simulé.	En V1.0 la porte s'ouvrait automatiquement. Incorrect : la porte est simulée par un bouton (appuyé = ouverte, relâché = fermée). RA0 lu en digital, pas en ADC.
V2.0	Moteur – Composant	IRF520 remplacé par driver L293D	Le MOSFET IRF520 est défaillant dans Proteus (Gate flottante = moteur toujours ON au démarrage). L293D est fiable et supporté par Proteus.
V2.0	Moteur – Brochage	IN1=RC0, IN2=RC1, EN=RC2 (CCP1)	Nouveau brochage avec le L293D. IN1/IN2 gèrent la direction, EN gère la vitesse via PWM CCP1.
V2.0	EEPROM I2C	Ajout SCL=RC3, SDA=RC4, adresse 0x50	Non mentionné en V1.0. Nécessaire pour la sauvegarde des trajets et du compteur.
V2.0	UART	TX=RC6, RX=RC7 confirmés (9600 bauds)	Déjà correct en V1.0. Confirmation explicite dans ce CDC.

Version	Section modifiée	Changement apporté	Justification
V2.0	Protocole UART	Ajout champ URG:x dans trame DATA	Distingue l'arrêt d'urgence (URG:1) d'une alarme normale (AL:1). Non documenté en V1.0.
V2.0	Protocole UART	Ajout 10 nouvelles commandes LabVIEW → PIC	CMD, STOP / START / PAUSE / ACK / MODE:AUTO / MODE:MAN / MOT:UP / MOT:DWN / AL:ON / RST:AL non documentées en V1.0.
V2.0	Protocole UART	Ajout trame réponse EEPROM	<DATA,TRAJ:x,LAST:x,VITESSE:x> non documentée en V1.0. Nécessaire pour afficher les données EEPROM dans LabVIEW.
V2.0	Protocole UART	Remplacement MOT:ON/OFF par MOT:UP/DWN	Cohérent avec le driver L293D qui gère la direction via IN1/IN2.
V2.0	Protocole UART	Suppression CMD,RST:EEP	Pas nécessaire dans le projet. Les données de l'EEPROM sont gardées comme historique.
V2.0	Protocole UART	DIR : DOWN remplacé par DWN	Cohérent avec le code MikroC qui envoie DWN dans la trame DATA.

7 Description du Fonctionnement – Version 2.0

Cette section remplace la section « Description du fonctionnement » de la Version 1.0. Toutes les valeurs sont corrigées et validées sur le schéma Proteus final.

7.1 Entrées Digitales

Composant	Broche	Résistance	Description
BP1 – Étage 0	RD0	Pull-down 10 kΩ	Appel cabine étage 0 (rez-de-chaussée). État HAUT = bouton pressé.
BP2 – Étage 1	RD1	Pull-down 10 kΩ	Appel cabine étage 1. État HAUT = bouton pressé.

Composant	Broche	Résistance	Description
BP3 – Étage 2	RD2	Pull-down 10 kΩ	Appel cabine étage 2. État HAUT = bouton pressé.
BP4 – Étage 3	RD3	Pull-down 10 kΩ	Appel cabine étage 3 (dernier étage). État HAUT = bouton pressé.
BP ACQ	RD4	Pull-down 10 kΩ	Acquittement alarme. Confirme la prise en charge d'une alerte active.
BP Urgence	RB6	Pull-down 10 kΩ	Arrêt d'urgence – Interruption externe IOC2. Coupe immédiatement le moteur.
BP5 – Alarme	RB7	Pull-down 10 kΩ	Déclenchement alarme manuelle – Interruption externe IOC3.

7.2 Capteurs

Capteur	Broche	Type lecture	Description
POT – Poids	RA1 / AN1	ADC 10 bits	Potentiomètre simulant le poids 0-900 kg. Appel : ADC_Read(1). Seuil surcharge : 693 kg (110% × 630 kg, norme EN81). Si dépassé : LED2 allumée + mouvement bloqué.
IR – Porte	RA0	Entrée digitale	Simule la détection de présence devant la porte. BOUTON POUSSOIR : maintenu appuyé = porte ouverte (IR=1) / relâché = porte fermée (IR=0). Lecture : PORTA.F0. Pas d'ouverture automatique.

7.3 Sorties – LEDs d'état

Composant	Broche	Résistance	Condition d'activation
LED1 – Verte	RA2	R1 – 220 Ω	Allumée si : cabine à l'arrêt ET porte libre (IR=0). Indique l'ascenseur disponible.
LED2 – Rouge	RA3	R2 – 220 Ω	Allumée si : poids ≥ 693 kg. Déplacement bloqué tant que surcharge active.

7.4 Moteur – Driver L293D (remplace IRF520)

Justification du remplacement :

Le MOSFET IRF520 utilisé en V1.0 est défaillant dans Proteus : au démarrage de la simulation, sa gate restait flottante, ce qui mettait le transistor en conduction tout de suite et lançait le moteur avant même que le code du PIC ne s'exécute. Le driver **L293D** a donc été choisi à la place, car il est bien pris en charge par Proteus et permet aussi de gérer le sens de rotation du moteur.

Signal	Broche PIC	Broche L293D	Rôle
IN1 – Marche avant	RC0	IN1 (pin 2)	Direction 1 : IN1=1, IN2=0 → rotation sens montée.
IN2 – Marche arrière	RC1	IN2 (pin 7)	Direction 2 : IN1=0, IN2=1 → rotation sens descente.
EN – Enable/PWM	RC2 (CCP1)	EN1 (pin 1)	Activation du driver + contrôle vitesse PWM. EN=0 = moteur arrêté.

Tableau logique de commande :

EN (RC2)	IN1 (RC0)	IN2 (RC1)	État moteur
0	X	X	ARRÊT – moteur coupé
1	1	0	MARCHE AVANT (montée)
1	0	1	MARCHE ARRIÈRE (descente)
1	1	1	INTERDIT – court-circuit

7.5 EEPROM I2C – NM24C08 (ajout V2.0)

Non documenté en V1.0. Brochage complet :

Signal	Broche PIC	Pull-up	Description
SCL – Horloge	RC3	10 kΩ vers +5V	Horloge I2C 100 kHz (mode standard).
SDA – Données	RC4	10 kΩ vers +5V	Données bidirectionnelles I2C.

7.6 Complément EEPROM I²C – Version 2.0

L'EEPROM NM24C08 est accessible sur le bus I²C à l'adresse **0x50** (broches A0, A1, A2 reliées à GND).

Adresse EEPROM	Donnée stockée	Mise à jour
0x00 – 0x01	Compteur total de trajets (2 octets)	Incrémenté à chaque trajet terminé.
0x02	Dernier étage desservi (1 octet)	Mis à jour à chaque arrêt à un étage.
0x03	Vitesse maximale configurée (1 octet, en %)	Mis à jour lors de la réception d'une commande <CMD,SPD:X>.
0x04 – 0x05	Poids maximum configuré (2 octets, en kg)	Mis à jour lors de la réception d'une commande <CMD,PMAX:X>.

Justification : la V1.0 ne précisait pas l'adresse I²C du composant (0x50). Le tableau des adresses mémoire est conservé et corrigé : compteur sur 2 octets (0x00–0x01), poids sur 2 octets (0x04–0x05).

7.7 Communication UART – PIC ↔ LabVIEW (confirmation V2.0)

7.7.1 Trames PIC → LabVIEW

Trame	Description	Exemple
<DATA,ET:x,DIR:x,PT:x,PRT:x,AL:x,URG :x>	Envoyée toutes les 2s via Timer0. Contient l'état complet du système.	<DATA,ET:2,DIR:UP,PT:185,PRT:F,AL:0,URG :0>
<DATA,TRAJ:x,LAST:x,VITESSE:x>	Réponse du PIC après <CMD,GET:EEP>. Contient les données EEPROM.	<DATA,TRAJ:42,LAST:2,VITESSE:75>
<ACK,OK>	Réponse du PIC après réception de <CMD,ACK>. Système en marche, commande acceptée.	<ACK,OK>
<ACK,ERR>	Réponse du PIC après réception de <CMD,ACK>. Commande refusée (urgence active, surcharge).	<ACK,ERR>

7.7.2 Champs de la trame DATA

Champ	Signification	Valeurs possibles
ET	Étage actuel de la cabine	0, 1, 2, 3
DIR	Direction du déplacement	UP = montée, DWN = descente, STP = arrêté
PT	Poids total dans la cabine (kg)	Valeur numérique (ex : 185)
PRT	État de la porte	O = ouverte, F = fermée
AL	Alarme active	0 = aucune alarme, 1 = alarme active
URG	Arrêt d'urgence actif	0 = fonctionnement normal, 1 = urgence déclenchée

7.7.3 Trames LabVIEW → PIC

Trame	Description	Exemple	Mode
<CMD,CALL:X>	Appel cabine à l'étage X (0 à 3).	<CMD,CALL:3>	Auto
<CMD,STOP>	Arrêt d'urgence immédiat du moteur.	<CMD,STOP>	Les deux
<CMD,PAUSE>	Arrête l'affichage en direct des trajets dans le tableau historique.	<CMD,PAUSE>	Les deux
<CMD,START>	Démarre l'affichage en direct des trajets dans le tableau historique.	<CMD,START>	Les deux
<CMD,ACK>	Acquittement LabVIEW vers PIC après réception d'une trame.	<CMD,ACK>	Les deux
<CMD,MODE:AUTO>	Basculer le système en mode automatique.	<CMD,MODE:AUTO>	Les deux
<CMD,MODE:MAN>	Basculer le système en mode manuel.	<CMD,MODE:MAN>	Les deux

Trame	Description	Exemple	Mode
<CMD,PMAX:X>	Envoi du poids maximum configuré (X en kg).	<CMD,PMAX:630>	Les deux
<CMD,SPD:X>	Envoi de la vitesse moteur (X en %). Contrôle PWM via EN du L293D.	<CMD,SPD:75>	Manuel
<CMD,DOOR:O>	Commande d'ouverture manuelle de la porte.	<CMD,DOOR:O>	Manuel
<CMD,DOOR:F>	Commande de fermeture manuelle de la porte.	<CMD,DOOR:F>	Manuel
<CMD,MOT:UP>	Marche avant — montée cabine. IN1=1, IN2=0 sur L293D.	<CMD,MOT:UP>	Manuel
<CMD,MOT:DWN>	Marche arrière — descente cabine. IN1=0, IN2=1 sur L293D.	<CMD,MOT:DWN>	Manuel
<CMD,AL:ON>	Déclenchement d'une alarme depuis LabVIEW.	<CMD,AL:ON>	Les deux
<CMD,RST:AL>	Réinitialisation de l'alarme active.	<CMD,RST:AL>	Les deux
<CMD,GET:EEP>	Demande de lecture des données EEPROM au PIC.	<CMD,GET:EEP>	Les deux

7.7.4 Justification des ajouts par rapport à la V1.0

Ajout / Correction	Justification
Champ URG dans DATA	Ajouté en V2.0 : distingue l'arrêt d'urgence (URG:1) d'une alarme (AL:1). Ça permet à LabVIEW de savoir exactement quel type d'événement a été reçu.
<DATA,TRAJ:x,LAST:x,VITESSE:x>	Non documenté en V1.0 : trame de réponse nécessaire après <CMD,GET:EEP> pour afficher les données EEPROM dans LabVIEW.

Ajout / Correction	Justification
<CMD,STOP>	Ajouté en V2.0 : arrêt d'urgence déclenché depuis LabVIEW en plus du bouton physique AU.
<CMD,PAUSE>	Ajouté en V2.0 : arrête l'affichage en direct des trajets dans le tableau historique côté LabVIEW. Le PIC continue à fonctionner normalement.
<CMD,START>	Ajouté en V2.0 : démarre l'affichage en direct des trajets dans le tableau historique côté LabVIEW. Le PIC continue à fonctionner normalement.
<CMD,ACK>	Ajouté en V2.0 : commande envoyée par LabVIEW vers le PIC pour vérifier l'état du système. Le PIC répond <ACK,OK> si tout est normal ou <ACK,ERR> si une urgence ou surcharge est active.
<CMD,MODE:AUTO/MAN>	Ajouté en V2.0 : bascule entre mode automatique et mode manuel depuis LabVIEW.
<CMD,MOT:UP/DWN>	Remplace <CMD,MOT:ON/OFF> : cohérent avec le driver L293D qui gère la direction via IN1/IN2. UP = montée, DWN = descente.
<CMD,AL:ON>	Non documenté en V1.0 : le champ AL existait dans la trame DATA mais aucune commande permettant de forcer AL=1 depuis LabVIEW n'était définie.
<CMD,RST:AL>	Non documenté en V1.0 : aucune commande permettant de remettre AL=0 depuis LabVIEW n'était définie dans le protocole.
DIR : DWN	Remplace DOWN : cohérent avec le code MikroC qui envoie DWN dans la trame DATA.
<CMD,RST:EEP>	Supprimé en V2.0 : la remise à zéro du compteur EEPROM n'est pas nécessaire dans le projet. Les données sont conservées comme historique.

7.8 Tableau Récapitulatif Complet du Brochage – Version 2.0

Broche PIC	Signal	Direction	Fonction
RA0	IR Porte	Entrée digitale	Capteur IR simulé – bouton poussoir (appuyé=ouvert)
RA1 / AN1	POT Poids	Entrée ADC	Poids cabine 0–900 kg – ADC_Read(1)
RA2	LED1 Verte	Sortie	Porte ouverte / ascenseur disponible
RA3	LED2 Rouge	Sortie	Surcharge détectée (≥ 693 kg)
RB0–RB3	LCD D4–D7	Sortie	Bus données LCD 16x2 mode 4 bits
RB4	LCD RS	Sortie	Register Select LCD
RB5	LCD EN	Sortie	Enable LCD
RB6	Urgence	Entrée IOC2	Arrêt d'urgence – interruption externe
RB7	BP5 Alarme	Entrée IOC3	Alarme manuelle – interruption externe
RC0	L293D IN1	Sortie	Direction moteur – Marche avant (montée)
RC1	L293D IN2	Sortie	Direction moteur – Marche arrière (descente)
RC2 / CCP1	L293D EN	Sortie PWM	Enable + vitesse moteur via PWM
RC3	SCL I2C	Sortie	Horloge EEPROM NM24C08
RC4	SDA I2C	Entrée/Sortie	Données EEPROM NM24C08
RC6	UART TX	Sortie	Émission vers LabVIEW – 9600 bauds
RC7	UART RX	Entrée	Réception depuis LabVIEW – 9600 bauds
RD0	BP1 Étage 0	Entrée	Bouton appel étage 0 – pull-down 10 k Ω
RD1	BP2 Étage 1	Entrée	Bouton appel étage 1 – pull-down 10 k Ω

Broche PIC	Signal	Direction	Fonction
RD2	BP3 Étage 2	Entrée	Bouton appel étage 2 – pull-down 10 kΩ
RD3	BP4 Étage 3	Entrée	Bouton appel étage 3 – pull-down 10 kΩ
RD4	BP ACQ	Entrée	Acquittement alarme – pull-down 10 kΩ

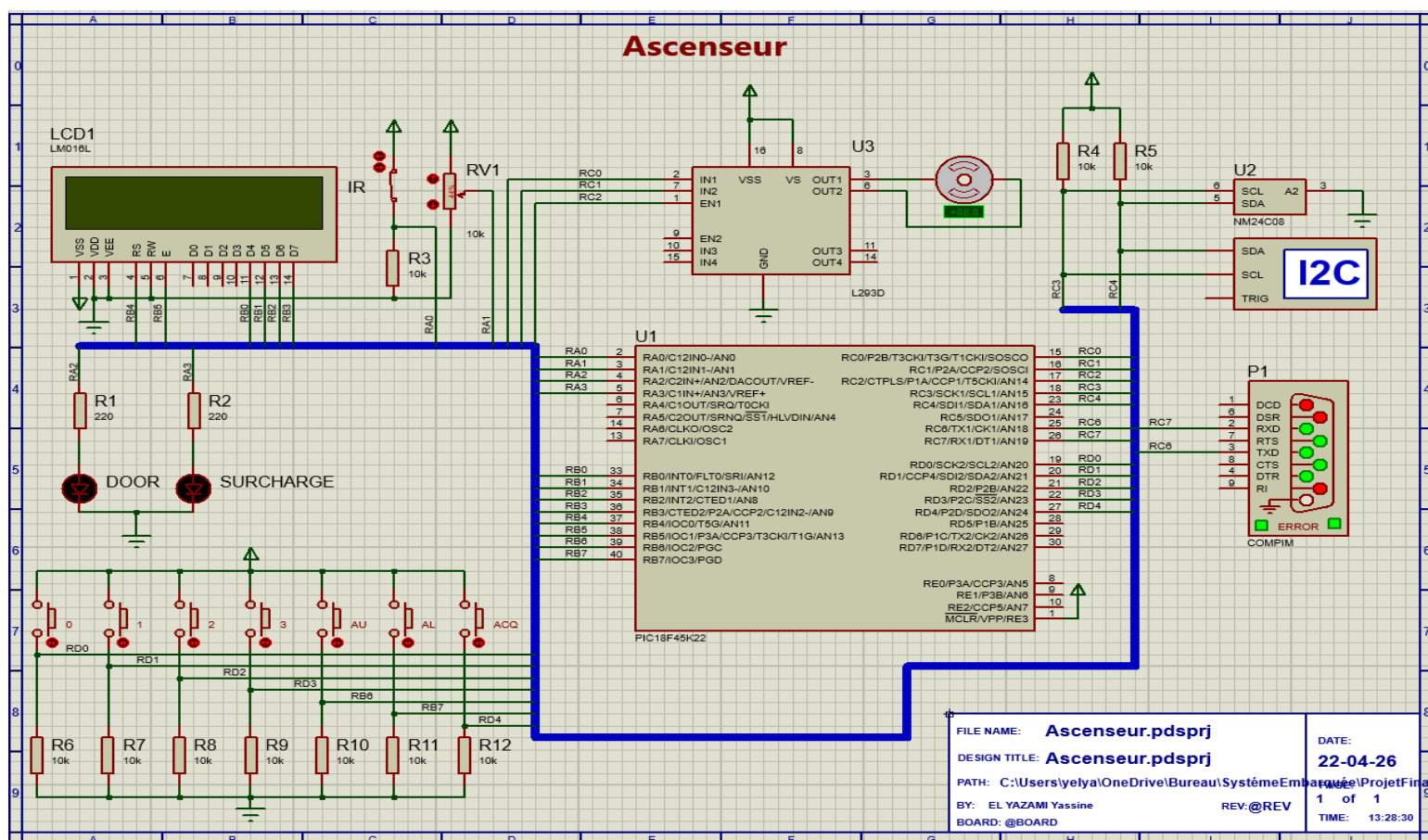


Schéma de simulation Proteus – Version 2.0

CDC Version 3.0

Mise à jour du protocole de communication et de la trame DATA

8 Historique des Révisions

Ce tableau liste l'ensemble des modifications apportées entre la Version 2.0 et la Version 3.0, avec la justification de chaque ajout et chaque suppression.

Version	Section modifiée	Changement apporté	Justification
V3.0	Protocole UART	Ajout <CMD,MOT:STOP>	En version 2.0, seules les commandes MOT:UP et MOT:DWN permettaient de faire tourner le moteur en mode manuel. Il n'y avait pas de commande prévue pour l'arrêter sans passer par l'arrêt d'urgence <CMD,STOP>. La commande <CMD,MOT:STOP> met le signal EN du driver L293D à 0, ce qui arrête immédiatement le moteur sans déclencher les routines d'alarme.
V3.0	Protocole UART	Ajout <CMD,EEP:RST>	Supprimée en V2.0 sous le nom <CMD,RST:EEP>, cette commande est réintroduite en V3.0 sous son nouveau nom. Elle remet à zéro les données sauvegardées en EEPROM : le compteur total de trajets, le dernier étage et la vitesse maximale, soit les mêmes données que celles retournées par <CMD,GET:EEP>.
V3.0	Protocole UART	Suppression <CMD,START>	Cette commande était présente dans la version 2.0 pour afficher le tableau historique dans LabVIEW. Finalement, cet affichage est géré par LabVIEW lui-même grâce à une structure événementielle. Le PIC n'a donc aucun rôle dans cette fonction. La commande a été retirée car elle n'était pas utile et compliquait le protocole pour rien.
V3.0	Protocole UART	Suppression <CMD,PAUSE>	Pour la même raison que <CMD,START>, l'arrêt de l'affichage en direct est géré directement dans LabVIEW par une structure événementielle. Le PIC n'intervient pas dans cette fonction, qui concerne uniquement le HMI.

Version	Section modifiée	Changement apporté	Justification
V3.0	Trame DATA	Ajout champ NB	NB est le compteur des trajets de la session en cours. Il augmente après chaque trajet terminé et revient à 0 au redémarrage du PIC. Il est séparé du compteur total enregistré dans l'EEPROM, qui lui reste conservé même après un redémarrage. Comme NB est envoyé dans la trame DATA, LabVIEW peut afficher directement le nombre de trajets réalisés depuis le dernier démarrage, sans avoir besoin d'une commande en plus.
V3.0	Trame DATA	Ajout champ PWM	Le rapport cyclique du moteur, en %, n'était pas envoyé à LabVIEW dans la version 2.0. En l'ajoutant dans la trame DATA, on peut afficher la vitesse réelle du moteur sur la face avant. C'est surtout utile en mode manuel pour vérifier que la commande <CMD,SPD:X> a bien été prise en compte par le PIC.
V3.0	Trame DATA	Ajout champ TPS	Ce champ indique la durée du dernier trajet terminé, en secondes. Elle est mesurée par le Timer0 entre le départ, quand le moteur se met en marche, et l'arrivée à l'étage cible, quand le moteur s'arrête. La valeur est mise à jour à la fin de chaque trajet, puis envoyée dans la trame DATA. Elle est ensuite affichée dans le tableau historique de LabVIEW.

9 Description du Fonctionnement – Version 3.0

Cette section remplace et complète la section 7 de la Version 2.0. Seules les parties modifiées sont redécrites. Le reste du projet (brochage, interruptions, EEPROM, face avant) reste conforme aux spécifications de la Version 2.0.

9.1 Trame DATA – Format mis à jour

<DATA,ET:%d,DIR:%s,PT:%d,PRT:%s,AL:%d,URG:%d,NB:%d,PWM:%d,TPS:%d>

Exemple :

<DATA,ET:2,DIR:UP,PT:185,PRT:F,AL:0,URG:0,NB:10,PWM:75,TPS:5>

Description complète de tous les champs :

Champ	Signification	Valeurs possibles
ET	Étage actuel de la cabine	0, 1, 2, 3

Champ	Signification	Valeurs possibles
DIR	Direction du déplacement	UP = 1, DWN = 2, STP = 0
PT	Poids total dans la cabine (kg)	Valeur numérique
PRT	État de la porte	O = ouverte, F = fermée
AL	Alarme active	0 = aucune alarme, 1 = alarme active
URG	Arrêt d'urgence actif	0 = fonctionnement normal, 1 = urgence déclenchée
NB	Compteur de trajets de la session en cours. Incrémenté après chaque trajet terminé. Remis à 0 au redémarrage du PIC. Différents du compteur EEPROM.	Valeur numérique — NOUVEAU V3.0
PWM	Rapport cyclique du moteur en cours (vitesse en %)	0 à 100 — NOUVEAU V3.0
TPS	Durée du dernier trajet terminé en secondes (mesuré par Timer0 du départ à l'arrivée à l'étage cible)	Valeur numérique — NOUVEAU V3.0

9.2 Trames PIC → LabVIEW

Trame	Description	Exemple
<DATA,ET:x,DIR:x,PT:x,PRT:x,AL:x,URG:x,NB:x,PWM:x,TPS:x>	Envoyée toutes les 2s via Timer0. Contient l'état complet du système, incluant les 3 nouveaux champs NB, PWM et TPS.	<DATA,ET:2,DIR:UP,PT:185,PRT:F,AL:0,URG:0,NB:10,PWM:75,TPS:5>
<DATA,TRAJ:x,LAST:x,VITESSE:x>	Réponse du PIC après <CMD,GET:EEP>. Contient les données lues depuis l'EEPROM.	<DATA,TRAJ:42,LAST:2,VITESSE:75>
<ACK,OK>	Acquittement positif : commande reçue et exécutée correctement.	<ACK,OK>

Trame	Description	Exemple
<ACK,ERR>	Acquittement négatif : commande refusée (urgence active, surcharge).	<ACK,ERR>

9.3 Trames LabVIEW → PIC – Tableau complet Version 3.0

Les commandes <CMD,START> et <CMD,PAUSE> sont supprimées du protocole. Les commandes <CMD,MOT:STOP> et <CMD,EEP:RST> sont ajoutées. Toutes les autres commandes de la Version 2.0 sont conservées sans modification.

Trame	Description	Exemple	Mode
<CMD,CALL:X>	Appel cabine à l'étage X (0 à 3).	<CMD,CALL:3>	Auto
<CMD,STOP>	Arrêt d'urgence immédiat du moteur, depuis LabVIEW.	<CMD,STOP>	Les deux
<CMD,ACK>	Vérification état système. Le PIC répond <ACK,OK> si tout est normal, <ACK,ERR> si urgence ou surcharge active.	<CMD,ACK>	Les deux
<CMD,MODE:AUTO>	Basculer le système en mode automatique.	<CMD,MODE:AUTO>	Les deux
<CMD,MODE:MAN>	Basculer le système en mode manuel.	<CMD,MODE:MAN>	Les deux
<CMD,PMAX:X>	Envoi du poids maximum configuré (X en kg). Sauvegardé en EEPROM.	<CMD,PMAX:630>	Manuel
<CMD,SPD:X>	Vitesse moteur (X en %). Contrôle PWM via signal EN du L293D. Sauvegardé en EEPROM.	<CMD,SPD:75>	Manuel
<CMD,DOOR:O>	Commande d'ouverture manuelle de la porte.	<CMD,DOOR:O>	Manuel
<CMD,DOOR:F>	Commande de fermeture manuelle de la porte.	<CMD,DOOR:F>	Manuel

Trame	Description	Exemple	Mode
<CMD,MOT:UP>	Marche avant — montée cabine. IN1=1, IN2=0 sur L293D.	<CMD,MOT:UP>	Manuel
<CMD,MOT:DWN>	Marche arrière — descente cabine. IN1=0, IN2=1 sur L293D.	<CMD,MOT:DWN>	Manuel
<CMD,MOT:STOP>	Arrêt du moteur en mode manuel. Met EN=0 sur L293D. Arrêt propre sans déclencher les routines d'alarme. — NOUVEAU V3.0	<CMD,MOT:STOP>	Manuel
<CMD,AL:ON>	Déclenchement d'une alarme depuis LabVIEW.	<CMD,AL:ON>	Les deux
<CMD,RST:AL>	Réinitialisation de l'alarme active.	<CMD,RST:AL>	Les deux
<CMD,GET:EEP>	Demande de lecture des données EEPROM au PIC. Le PIC répond avec une trame <DATA,TRAJ:x,LAST:x,VITESSE:x>.	<CMD,GET:EEP>	Les deux
<CMD,EEP:RST>	Remet à zéro les trois valeurs de l'EEPROM : compteur total de trajets, dernier étage, vitesse maximale sauvegardée. — NOUVEAU V3.0	<CMD,EEP:RST>	Les deux

9.4 Justification des ajouts et suppressions par rapport à la Version 2.0

Ajout / Suppression	Justification
<CMD,MOT:STOP> (AJOUT)	En version 2.0, MOT:UP et MOT:DWN servaient à faire tourner le moteur dans un sens ou dans l'autre, mais il n'existait pas de commande pour l'arrêter en mode manuel. L'opérateur devait alors passer par <CMD,STOP>, ce qui lançait l'arrêt d'urgence et les routines d'alarme. Avec <CMD,MOT:STOP>, le signal EN du L293D passe à 0 : le moteur s'arrête tout de suite, sans déclencher l'alarme.
<CMD,EEP:RST> (AJOUT)	Supprimée en version 2.0 sous le nom <CMD,RST:EEP>, cette commande est réintroduite en version 3.0 avec son nouveau nom. Elle remet à zéro les

Ajout / Suppression	Justification
	données enregistrées dans l'EEPROM : le compteur total de trajets, le dernier étage et la vitesse maximale.
<CMD,START> (SUPPRESSION)	Cette commande était censée lancer l'affichage du tableau historique dans LabVIEW. En réalité, cette fonction est entièrement gérée côté LabVIEW par une Event Structure, sans intervention du PIC. Sa suppression permet donc de simplifier le protocole sans changer le fonctionnement du système.
<CMD,PAUSE> (SUPPRESSION)	Pour la même raison que <CMD,START>, l'arrêt de l'affichage en direct est géré directement dans LabVIEW par une Event Structure. Cette fonction est locale au HMI et ne nécessite donc aucun échange de données avec le PIC.
Champ NB dans DATA (AJOUT)	NB est un compteur de session. Il est incrémenté en mémoire RAM après chaque trajet terminé et revient à 0 à chaque redémarrage du PIC. Il est indépendant du compteur enregistré dans l'EEPROM, qui lui conserve sa valeur même après un redémarrage. En l'ajoutant à la trame DATA, LabVIEW peut afficher directement le nombre de trajets effectués depuis le démarrage actuel.
Champ PWM dans DATA (AJOUT)	La valeur du rapport cyclique PWM du moteur n'était pas envoyée à LabVIEW dans la version 2.0. Son ajout permet maintenant d'afficher directement la vitesse du moteur sur la face avant. C'est utile en mode manuel pour vérifier que la commande <CMD,SPD:X> a bien été prise en compte par le PIC.
Champ TPS dans DATA (AJOUT)	TPS correspond à la durée du dernier trajet effectué, exprimée en secondes. Le Timer0 commence à compter lorsque le moteur démarre et s'arrête lorsque l'étage demandé est atteint. Cette valeur est ensuite envoyée dans la trame DATA à la fin du trajet et affichée dans le tableau historique de LabVIEW, dans la colonne Durée.